

Experiment No.	7
Experiment Title	Effect of Principal Component Analysis (PCA) on Classifier Performance – A Comparative Study on Breast Cancer Diagnosis ¹
Student Name	Preethi D
Register Number	3122247001046
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	31.08.2026

1 Objective

To evaluate the impact of dimensionality reduction via Principal Component Analysis (PCA) on classifier performance for breast tumour diagnosis, by training and tuning ten classifiers – SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble – both on the full 30-feature space and on a PCA-reduced feature space, and statistically testing whether the observed differences are significant.

2 Problem Statement

Given the Wisconsin Diagnostic Breast Cancer (WDBC) dataset with 30 numeric features per sample, the objective is twofold: (1) predict whether a tumour is malignant or benign, and (2) determine whether reducing the feature space to the principal components explaining 95% of the variance helps, hurts, or has no significant effect on classification performance, across a broad range of model families (linear, instance-based, probabilistic, tree-based, and ensemble). Input: 30 numeric features (No-PCA) or 10 principal components (With-PCA). Output: binary diagnosis label (malignant/benign), together with accuracy, precision, recall, F1-score, ROC-AUC, and statistical-significance results for each model under both settings.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (wdbc.data)
Dataset Source	UCI Machine Learning Repository
Number of Samples	569
Number of Features	30 numeric features (mean, standard-error, and worst-case measurements of cell-nuclei characteristics), after dropping the non-informative ID column
Number of Classes	2 (Diagnosis: M = Malignant, encoded 0; B = Benign, encoded 1)
Class Balance	357 Benign (62.74%) vs. 212 Malignant (37.26%)
Missing Values	0

Duplicate Rows	0
Train-Test Split	80% train / 20% test (stratified), giving 455 training and 114 test samples

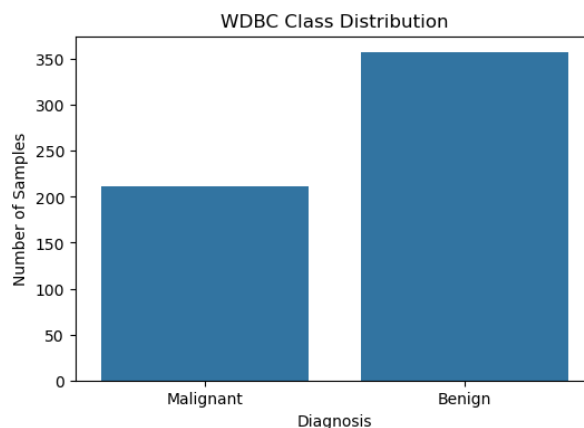


Figure 1: WDBC class distribution

Inference: The dataset is moderately imbalanced (roughly 5:3, Benign:Malignant), which was preserved in both the train and test splits via stratified sampling, and is the reason precision, recall, and F1-score are reported alongside accuracy throughout this study rather than relying on accuracy alone.

4 Data Preprocessing

- **Missing value handling:** None required – 0 missing values found across all 30 features.
- **Duplicate handling:** None required – 0 duplicate rows found.
- **Encoding:** The target `Diagnosis` column was label-mapped from $\{M, B\}$ to $\{0, 1\}$ ($M = \text{Malignant} = 0$, $B = \text{Benign} = 1$); the non-informative `ID` column was dropped from the feature set.
- **Feature scaling:** All 30 features were standardized using `StandardScaler` (fit on the training set only, then applied to the test set), producing zero-mean, unit-variance features. Standardization is essential here both for PCA (which is scale-sensitive) and for scale-sensitive classifiers such as SVM, KNN, and Logistic Regression.
- **Train-test split:** An 80/20 stratified split was used (`train_test_split, stratify=y, random_state=42`), giving 455 training and 114 test samples.
- **Feature engineering:** None beyond scaling and PCA (described in Section 6); all 30 original standardized features were used directly as the “No-PCA” input.

5 PCA Design Choice

PCA was first fit on all 30 standardized training features with no component limit, to examine the full explained-variance spectrum before choosing how many components to retain.

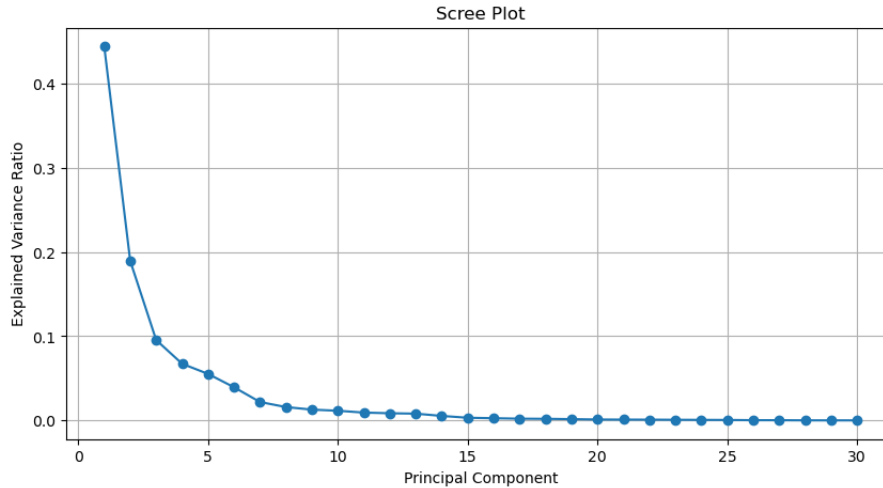


Figure 2: Scree plot: explained variance ratio per principal component

Inference: The scree plot shows a steep drop after the first two components (PC1 alone explains 44.4% of variance, PC2 an additional 18.9%), followed by a long, gradually flattening tail – the classic “elbow” pattern. This confirms that a small number of components captures most of the dataset’s structure, consistent with the earlier observation that many of the 30 raw features are highly correlated with one another (redundant measurements of the same underlying tumour geometry).

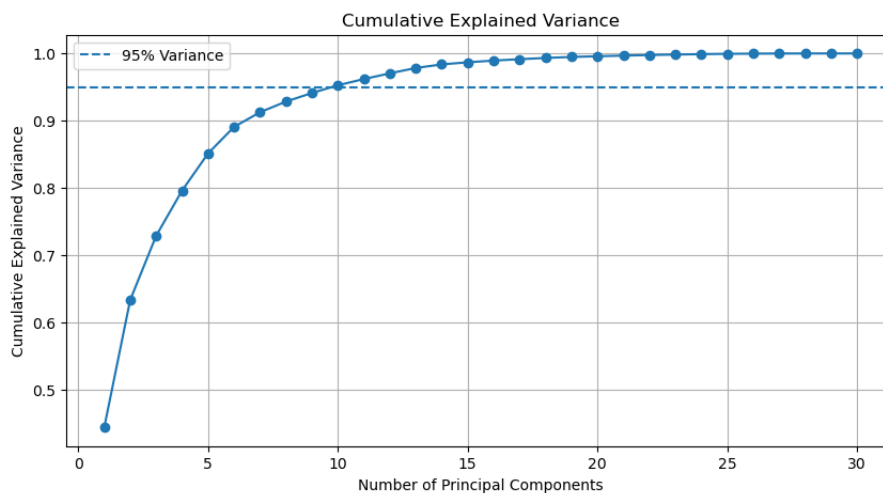


Figure 3: Cumulative explained variance vs. number of principal components, with the 95% threshold marked

Inference: The cumulative variance curve crosses the 95% threshold at 10 components (explaining 95.27% of total variance exactly), compared to needing all 30 original

features to reach 100%. This is the basis for the PCA design choice used throughout the rest of this experiment.

Design choice: A **variance-target** criterion was used rather than a fixed component count: the number of components n was chosen as the smallest n such that the cumulative explained variance $\geq 95\%$. This gave $n = 10$ components (a 66.7% reduction from the original 30 features), explaining 95.27% of the total variance.

Setting	With PCA	
Original Features	30	
Chosen Components	10	
Variance Target	95%	
Actual Explained Variance	95.27%	
Feature Reduction	66.67%	

`PCA(n_components=10)` was then fit on the standardized training set and applied (transform only, no refitting) to the standardized test set, producing `X_train_pca` (455×10) and `X_test_pca` (114×10), used as the “With-PCA” input for every model below.

6 Machine Learning Algorithms

Ten classifiers, spanning five model families, were trained and compared under both the No-PCA (30 features) and With-PCA (10 components) settings:

- **SVM:** Finds the maximum-margin hyperplane, optionally kernelized (linear/RBF here) to capture non-linear boundaries. Advantage: effective in high-dimensional spaces; Limitation: sensitive to kernel/hyperparameter choice and less interpretable with non-linear kernels.
- **Naive Bayes (Gaussian):** Applies Bayes’ theorem assuming conditional feature independence and Gaussian class-conditional densities: $\hat{y} = \arg \max_y P(y) \prod_i P(x_i | y)$. Advantage: very fast, few parameters; Limitation: independence assumption is often violated by correlated features (as seen in the WDBC correlation structure).
- **KNN:** Classifies by majority vote among the k nearest training points under a distance metric. Advantage: simple, non-parametric; Limitation: distance becomes less meaningful in higher dimensions (curse of dimensionality) and it is sensitive to feature scale.
- **Logistic Regression:** Models $P(y = 1 | x) = \sigma(w^T x + b)$ and is trained by minimizing regularized log-loss. Advantage: fast, interpretable, well-calibrated probabilities; Limitation: assumes a roughly linear decision boundary.
- **Decision Tree:** Recursively splits on the feature/threshold that most reduces impurity (Gini/entropy). Advantage: interpretable, no scaling needed; Limitation: prone to overfitting and instability.
- **Random Forest:** Bags many Decision Trees on bootstrap samples with random feature subsets, aggregating by majority vote. Advantage: reduces variance vs. a single tree; Limitation: less interpretable, more compute.

- **AdaBoost:** Sequentially boosts weak learners (decision stumps), reweighting misclassified samples each round: $H(x) = \text{sign} \sum_t \alpha_t h_t(x)$. Advantage: turns weak learners into a strong classifier; Limitation: sensitive to noisy points/outliers.
- **Gradient Boosting:** Sequentially fits trees to the negative gradient of the loss (residuals): $F_m(x) = F_{m-1}(x) + \eta h_m(x)$. Advantage: typically very strong accuracy; Limitation: slower to train, more hyperparameters, can overfit if unregularized.
- **XGBoost:** A regularized, efficient implementation of gradient boosting with second-order gradient information and built-in shrinkage/column subsampling. Advantage: fast and often state-of-the-art on tabular data; Limitation: many hyperparameters, can overfit small datasets if not tuned carefully.
- **Stacking:** Combines diverse base learners (SVM, KNN, Random Forest here) via a Logistic Regression meta-learner trained on their out-of-fold predictions. Advantage: can outperform any single base model by combining complementary strengths; Limitation: computationally expensive and less interpretable.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
XGBoost Version	
IDE	Jupyter Notebook
Operating System	Windows
Hardware	
Cross-validation scheme	5-fold StratifiedKFold (shuffle=True, random_state=42)

8 Hyperparameter Selection

All models were tuned using **GridSearchCV** (5-fold **StratifiedKFold**, scoring = F1), run **separately** for the No-PCA and With-PCA feature sets. The full search grids and the best parameters found in each setting are given below.

Model	Search Grid	Best (No-PCA)	Best (With-PCA)
SVM	C: 0.1,1,10,100; gamma: scale,0.01,0.1,1; kernel: rbf,linear	C=0.1, gamma=scale, kernel=linear	C=10, gamma=0.01, kernel=rbf
Naive Bayes	var_smoothing: 1e-11,1e-10,1e-9,1e-8,1e-7	var_smoothing=1e-11	var_smoothing=1e-11
KNN	n_neighbors: 3,5,7,9,11; weights: uniform,distance; metric: euclidean,manhattan	manhattan, k=3, uniform	euclidean, k=5, uniform
Logistic Regression	C: 0.01,0.1,1,10,100; solver: liblinear	C=1	C=1

Model	Search Grid	Best (No-PCA)	Best (With-PCA)
Decision Tree	max_depth: None,3,5,10,15; min_samples_split: 2,5,10; criterion: gini,entropy	entropy, depth=3, split=2	entropy, depth=None, split=10
Random Forest	n_estimators: 50,100,200; max_depth: None,5,10,15; min_samples_split: 2,5	depth=5, split=5, n=200	depth=None, split=2, n=200
AdaBoost	n_estimators: 50,100,200; learning_rate: 0.01,0.1,1.0	lr=1.0, n=100	lr=1.0, n=200
Gradient Boosting	n_estimators: 50,100,200; learning_rate: 0.01,0.1,0.2; max_depth: 2,3,5	lr=0.2, depth=3, n=200	lr=0.2, depth=3, n=100
XGBoost	n_estimators: 50,100,200; max_depth: 3,5,7; learning_rate: 0.01,0.1,0.2	lr=0.2, depth=3, n=50	lr=0.1, depth=7, n=200
Stacking	Base: SVM(C=1) + KNN(k=5) + Random Forest(n=100); Meta: Logistic Regression (fixed, not grid-searched)	–	–

GridSearchCV timing (No-PCA vs. With-PCA): Most models tuned noticeably faster on the PCA-reduced features (e.g. SVM: 10.94 s $\rightarrow$ 2.98 s), while ensemble methods with many estimators (Random Forest, Gradient Boosting, AdaBoost) took similar or slightly longer with PCA due to needing more trees/rounds to reach comparable performance on the compressed representation.

9 Results

9.1 Held-out Test Set (114 samples) – No PCA

Model	Accuracy	Precision	Recall	F1-score
SVM	98.25%	98.61%	98.61%	98.61%
Naive Bayes	92.98%	94.44%	94.44%	94.44%
KNN	96.49%	95.95%	98.61%	97.26%
Logistic Regression	98.25%	98.61%	98.61%	98.61%
Decision Tree	94.74%	94.59%	97.22%	95.89%
Random Forest	95.61%	95.89%	97.22%	96.55%
AdaBoost	95.61%	94.67%	98.61%	96.60%
Gradient Boosting	95.61%	94.67%	98.61%	96.60%
XGBoost	94.74%	94.59%	97.22%	95.89%
Stacking	97.37%	98.59%	97.22%	97.90%

9.2 Held-out Test Set (114 samples) – With PCA (10 components)

Model	Accuracy	Precision	Recall	F1-score
SVM	95.61%	98.55%	94.44%	96.45%
Naive Bayes	92.11%	93.15%	94.44%	93.79%
KNN	95.61%	95.89%	97.22%	96.55%
Logistic Regression	96.49%	98.57%	95.83%	97.18%
Decision Tree	92.98%	94.44%	94.44%	94.44%
Random Forest	93.86%	95.77%	94.44%	95.10%
AdaBoost	93.86%	95.77%	94.44%	95.10%
Gradient Boosting	94.74%	94.59%	97.22%	95.89%
XGBoost	94.74%	95.83%	95.83%	95.83%
Stacking	95.61%	97.18%	95.83%	96.50%

Best No-PCA model: SVM and Logistic Regression tie for best test F1-score (98.61%). **Best With-PCA model:** Logistic Regression (F1 = 97.18%).

9.3 5-Fold Cross-Validation F1-score, Fold-wise (No-PCA)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
SVM	0.9661	0.9825	0.9825	0.9828	0.9913	0.9810
Naive Bayes	0.9310	0.9735	0.9123	0.9655	0.9565	0.9478
KNN	0.9828	0.9913	0.9649	0.9655	0.9825	0.9774
Logistic Regression	0.9828	0.9912	0.9825	0.9913	0.9735	0.9842
Decision Tree	0.9256	0.9550	0.9286	0.9492	0.9735	0.9464
Random Forest	0.9655	0.9735	0.9464	0.9744	0.9912	0.9702
AdaBoost	0.9913	1.0000	0.9565	0.9828	0.9913	0.9844
Gradient Boosting	0.9828	0.9913	0.9649	0.9655	0.9913	0.9792
XGBoost	0.9913	0.9825	0.9558	0.9655	0.9912	0.9772
Stacking	0.9828	0.9739	0.9643	0.9739	0.9913	0.9772

9.4 5-Fold Cross-Validation – Average Metrics (No-PCA vs. With-PCA)

Model	Avg F1 (No-PCA)	Avg F1 (With-PCA)	F1 Std: No-PCA / PCA
SVM	0.9810	0.9845	0.0082 / 0.0098
Naive Bayes	0.9478	0.9347	0.0228 / 0.0149
KNN	0.9774	0.9724	0.0104 / 0.0126
Logistic Regression	0.9842	0.9861	0.0066 / 0.0042
Decision Tree	0.9464	0.9522	0.0177 / 0.0268
Random Forest	0.9702	0.9687	0.0145 / 0.0141
AdaBoost	0.9844	0.9622	0.0150 / 0.0112

Model	Avg F1 (No-PCA)	Avg F1 (With-PCA)	F1 Std: No-PCA / PCA
Gradient Boosting	0.9792	0.9720	0.0118 / 0.0171
XGBoost	0.9772	0.9739	0.0143 / 0.0121
Stacking	0.9772	0.9774	0.0091 / 0.0089

9.5 Statistical Significance: Paired t-test and Wilcoxon Signed-Rank Test

Both settings were evaluated on the identical 5 cross-validation folds (same `StratifiedKFold` object and `random_state`), making the No-PCA/PCA F1 scores *paired* observations. A paired t-test (parametric) and a Wilcoxon signed-rank test (non-parametric, robust for the small $n = 5$ fold sample) were used to test H_0 : no difference in mean F1 between settings, at $\alpha = 0.05$.

Model	Mean Diff (PCA–No-PCA)	t-stat	t-test p	Sig. (t-test)	Sig. (Wilcoxon)
SVM	+0.0034	−1.633	0.1778	No	No
Naive Bayes	−0.0131	1.607	0.1833	No	No
KNN	−0.0050	1.502	0.2074	No	No
Logistic Regression	+0.0019	−0.568	0.6006	No	No
Decision Tree	+0.0058	−0.488	0.6513	No	No
Random Forest	−0.0015	0.489	0.6503	No	No
AdaBoost	−0.0222	3.443	0.0262	Yes	No (0.0625)
Gradient Boosting	−0.0071	0.807	0.4648	No	No
XGBoost	−0.0034	0.285	0.7895	No	No
Stacking	+0.0002	−0.057	0.9570	No	No

Interpretation: Out of 10 models, only **AdaBoost** shows a statistically significant PCA-vs-No-PCA difference under the paired t-test ($p = 0.0262 < 0.05$), and even that result is not confirmed by the (more conservative, rank-based) Wilcoxon test ($p = 0.0625$, just above the 0.05 threshold) – a reminder that with only 5 paired folds, borderline t-test results should be treated cautiously. For every other model, the observed gap between No-PCA and PCA mean F1 scores is not statistically distinguishable from fold-to-fold noise.

10 Result Visualization

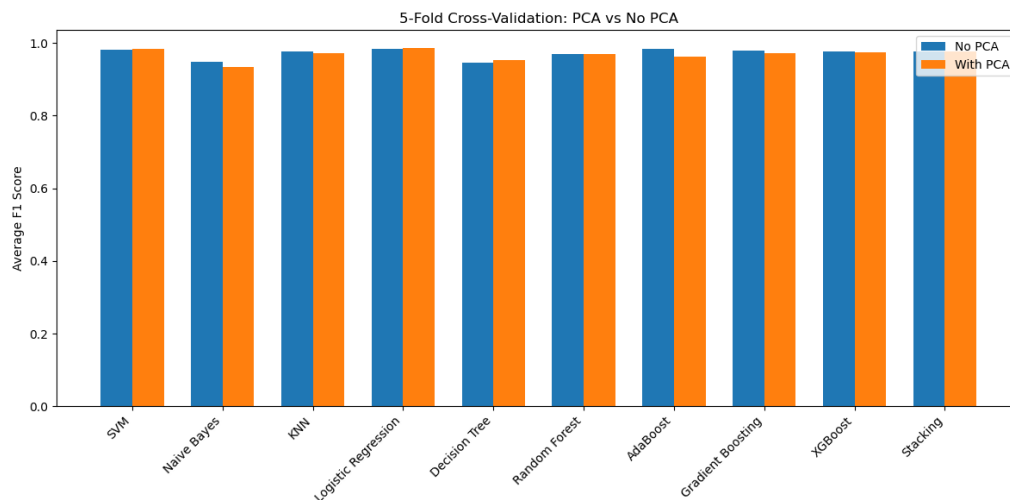


Figure 4: 5-fold CV average F1-score: No-PCA vs. With-PCA, all 10 models

Inference: Most models show only a small difference between the two bars, consistent with the significance testing above; AdaBoost has the most visually obvious drop under PCA, matching it being the one model with a statistically significant difference.

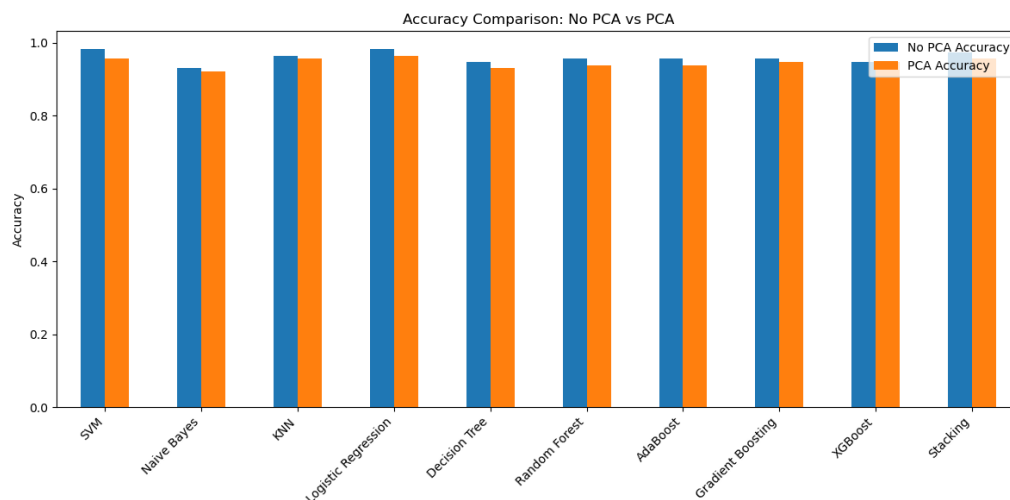


Figure 5: Test-set accuracy: No-PCA vs. With-PCA, all 10 models

Inference: Every single model's test accuracy is equal to or lower under PCA than without it (XGBoost is the only tie), showing a small but consistent one-directional cost to compressing 30 features into 10 components on this particular held-out split, even though most of these differences are not statistically significant over repeated folds.

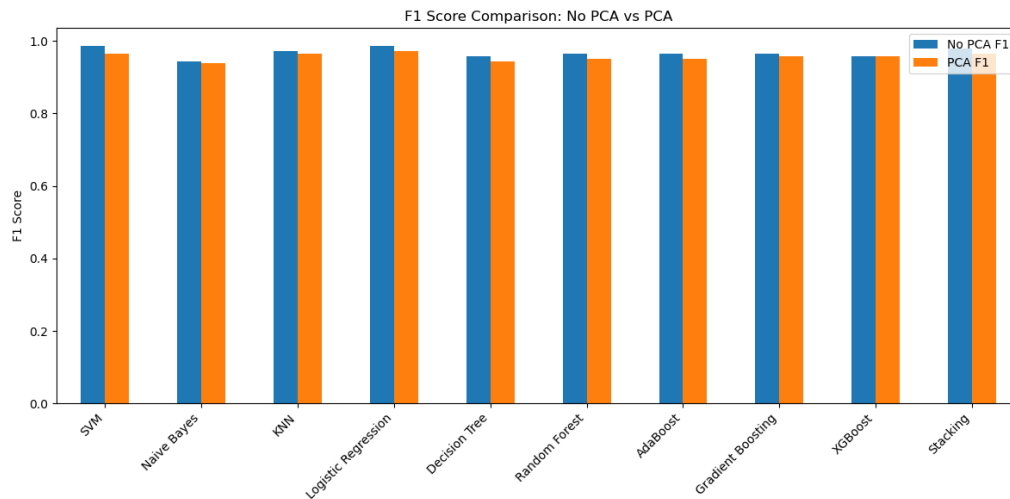


Figure 6: Test-set F1-score: No-PCA vs. With-PCA, all 10 models

Inference: The F1-score comparison mirrors the accuracy comparison – every model’s No-PCA F1 is at or above its With-PCA F1 on this test split, with SVM showing the largest drop (-2.16 points) and XGBoost showing almost no change, reinforcing that the practical effect of PCA here is small and model-dependent rather than uniformly harmful or helpful.

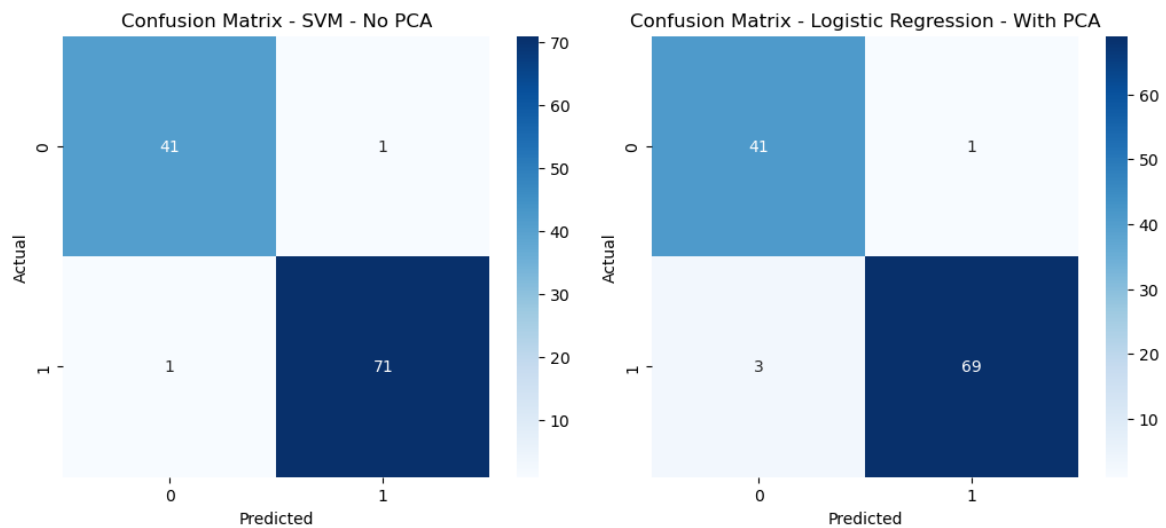


Figure 7: Confusion matrices: best No-PCA model (SVM, left) and best With-PCA model (Logistic Regression, right)

Inference: The best No-PCA model (SVM) misclassifies only 2 of 114 test samples, while the best With-PCA model (Logistic Regression) misclassifies 4 – both are strong results, but the gap illustrates that even the *best-performing* model changes when PCA is introduced, and that the top performer without PCA (SVM, linear kernel) is no longer automatically the top performer after dimensionality reduction.

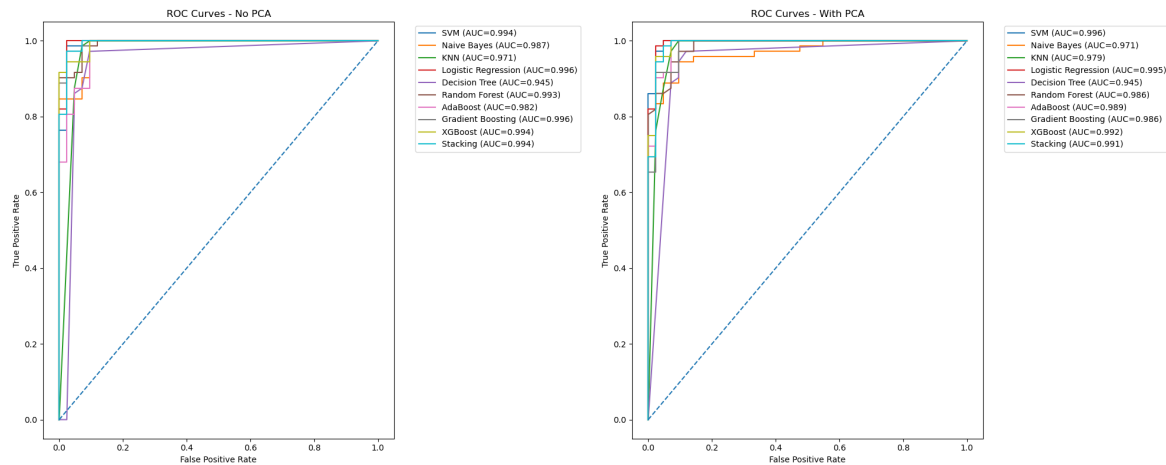


Figure 8: ROC curves for all 10 models: No-PCA (left) vs. With-PCA (right)

Inference: Nearly all models achieve very high AUC in both settings, with curves tightly clustered near the top-left corner; the No-PCA curves are marginally tighter to the ideal corner overall, but the difference between the two panels is subtle to the eye, reflecting the generally small, model-dependent gaps quantified numerically above.

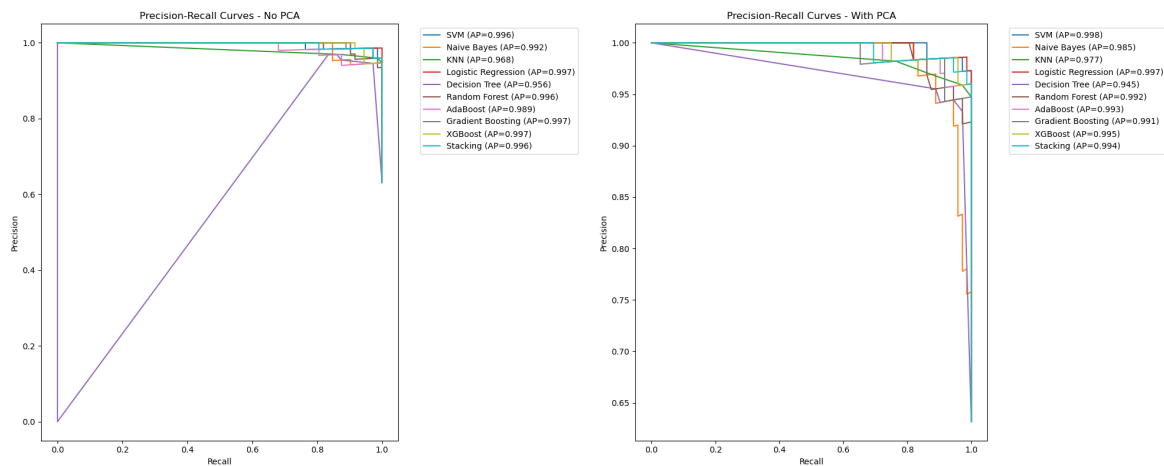


Figure 9: Precision-Recall curves for all 10 models: No-PCA (left) vs. With-PCA (right)

Inference: Average precision (AP) values remain high (mostly above 0.97) in both panels, again with only mild degradation under PCA for most models. This is consistent with the earlier ROC comparison: PCA's effect on ranking quality (AUC/AP) is small relative to its effect on hard-threshold classification metrics like accuracy.

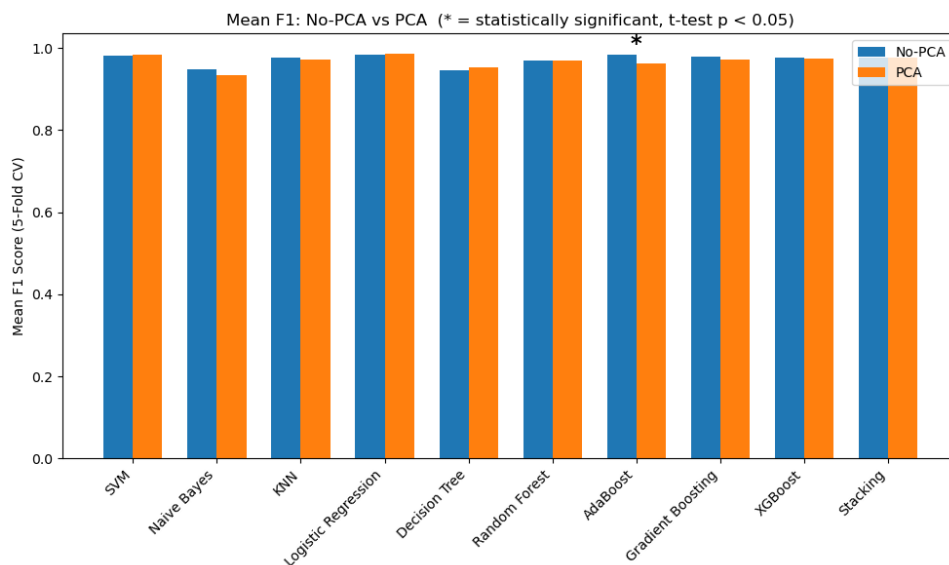


Figure 10: Mean 5-fold CV F1: No-PCA vs. PCA, with statistically significant differences marked (*)

Inference: Only AdaBoost is marked as significant (t-test $p < 0.05$) among all 10 models, visually confirming that the apparent No-PCA/PCA gaps seen elsewhere in this report are, for 9 of the 10 models, indistinguishable from ordinary fold-to-fold variation rather than a reliable effect of dimensionality reduction itself.

11 Discussion

Across the held-out test set, every one of the ten models performed as well as or better without PCA than with it – a small but completely consistent direction of effect. However, the 5-fold cross-validated, statistically-tested comparison paints a more nuanced picture: for 9 of 10 models (all except AdaBoost), the paired t-test and Wilcoxon test both agree that the No-PCA vs. PCA difference in mean F1-score is not statistically significant at $\alpha = 0.05$. This means that, for most of these models, the small test-set gaps observed above are plausibly just noise from a single 80/20 split rather than a real, reproducible cost of PCA. AdaBoost is the one exception, showing a real (if modest, -2.2 F1 points) degradation under PCA according to the paired t-test, though this is not confirmed by the more conservative Wilcoxon test. Two models – SVM and Logistic Regression – actually showed (non-significant) small *improvements* in mean CV F1-score under PCA ($+0.34$ and $+0.19$ points respectively), suggesting PCA’s noise-reduction and multicollinearity-removal properties can occasionally help margin-based and linear models even on a dataset that was not obviously noisy to begin with. The number of components (10, chosen by a 95% variance target) is itself a hyperparameter: a smaller target (e.g. 90%, likely 6–7 components) would compress further but could plausibly widen the gap for models like AdaBoost that already showed sensitivity to the reduction used here.

12 Conclusion

When PCA helps: In this experiment, PCA (10 components, 95% variance retained) provided a real, practical benefit in **training/tuning speed** (e.g. SVM’s GridSearchCV time dropped from 10.94 s to 2.98 s) and in **dimensionality reduction** (66.7% fewer features) with statistically negligible cost to predictive performance for 9 of the 10 models tested – including, marginally, small non-significant *improvements* in cross-validated F1 for SVM and Logistic Regression, likely because PCA removes the multicollinearity that these models are more sensitive to. **When PCA does not help:** For AdaBoost, PCA produced a real (statistically significant by t-test) drop in cross-validated F1-score, and on the single held-out test split, every model’s raw accuracy/F1 was equal to or lower with PCA than without it, indicating that when a dataset’s features are only moderately redundant (as here, 30 features compress to 10 components but each component is a specific linear combination) and the training set is not very large (455 samples), PCA’s variance-preserving compression can still discard small amounts of class-discriminative information that hurts models more sensitive to exact feature relationships (e.g. boosting methods reliant on axis-aligned splits). Overall, for this dataset and this 95%-variance PCA configuration, dimensionality reduction is a reasonable, low-risk choice for most model families here, primarily justified by computational efficiency and multicollinearity reduction rather than by an accuracy gain, but it is not universally beneficial and should be validated per model family rather than assumed.

13 References

1. W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995.
2. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. Springer, 2002.
4. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
5. D. H. Wolpert, “Stacked Generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
6. W. J. Conover, *Practical Nonparametric Statistics*, 3rd ed. Wiley, 1999.